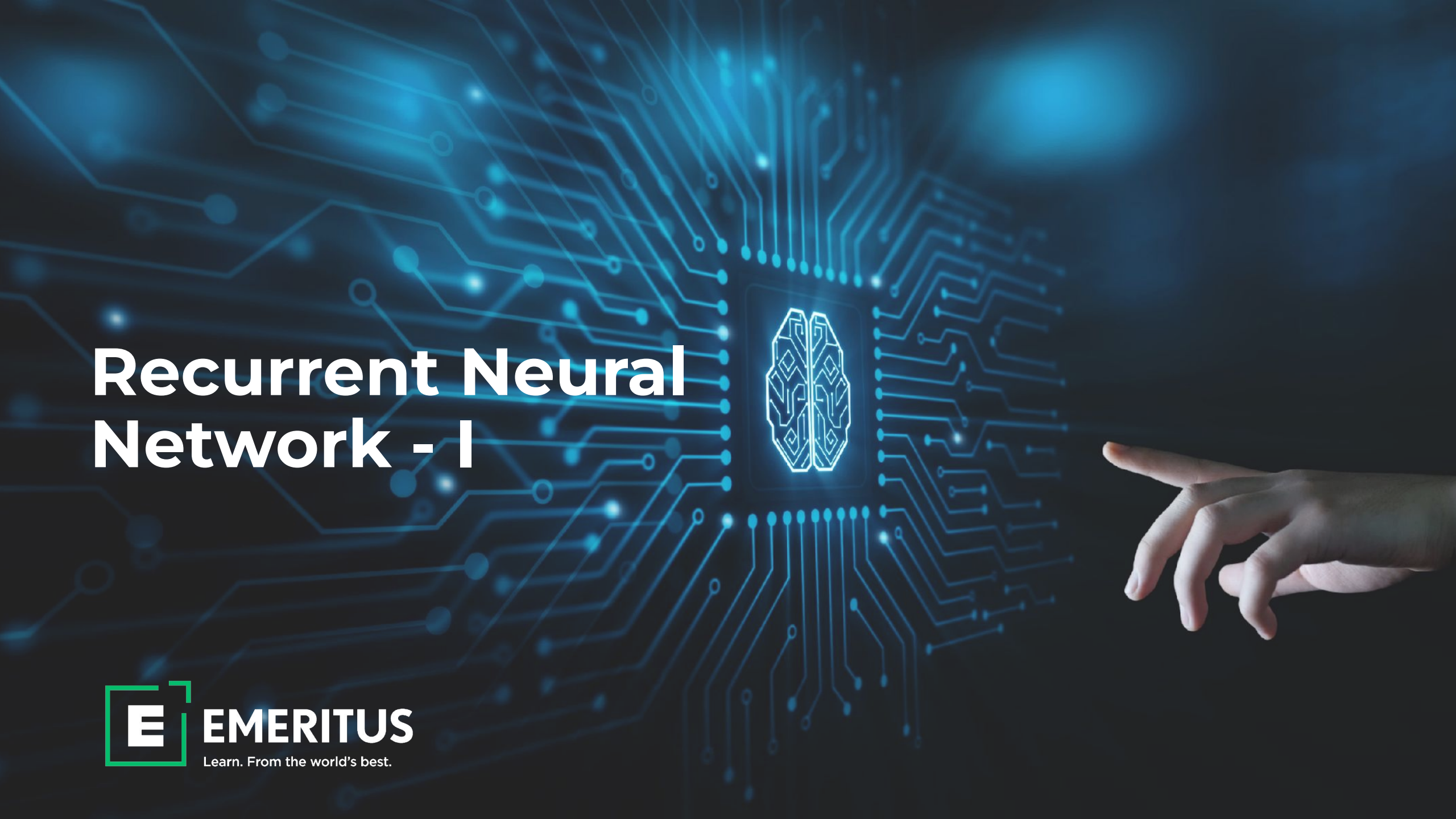

EMERITUS
Learn. From the world's best.

Recurrent Neural Network - I



EMERITUS
Learn. From the world's best.

RNN: Recap

RNNs: Recap

Neural Networks For Sequence Data

1. **Definition**
   Neural networks created to process sequences of data

2. **Applications**
   Used in text generation, translation, forecasting, and speech recognition

3. **Key feature**
   Remembers previous inputs to guide current output

Challenges in Training RNNs

Key Limitations To Be Aware Of

Gradient issues

Vanishing and exploding gradients in backpropagation

Long-term dependencies

Difficulty capturing patterns across many time steps

Overfitting and computation

Prone to overfitting and costly for long sequences

Gated RNN Variants

Popular Improvements On Standard RNNs

LSTM
Manages long-term dependencies using forget, input and output gates

GRU
A lighter version with fewer gates, quicker to train

Usage
LSTM suits complex sequences, GRU suits faster training

Optimisation Techniques

Methods To Improve RNN Training

Gradient clipping

Restricts gradient size to avoid explosion

Batch Normalisation

Normalises activations across batches to stabilise training

Dropout

Randomly removes units during training to reduce overfitting

Best Practices for RNNs

Tips For Effective Training

1. Data Preprocessing

- **Normalise/Standardise Data**: Bring input features to zero mean and unit variance for smoother and more stable training
- **Sequence Padding/Truncation**: Make all sequences the same length using padding for shorter ones or truncation for longer ones. Tools like Keras pad_sequences() help with this
- **Remove Outliers**: Detect and manage outliers, especially in time series, to avoid skewed learning and improve model robustness

2. Batching and Sequence Handling

- **Use Time-Major Batches**: Use the format (sequence_length, batch_size, features) instead of (batch_size, sequence_length, features) for better computational performance
- **Shuffling Data**: Shuffle sequences when they are independent to improve generalisation. For time series, maintain order to preserve temporal patterns
- **Stateful vs. Stateless LSTM**: Choose stateful LSTM when dependencies span across batches. Remember to reset states when moving between unrelated sequences

Best Practices for RNNs

Tips For Effective Training

3. Model Architecture

- **Stacked RNNs**: Use multiple RNN layers (stacked LSTM/GRU) to capture complex patterns. Set return_sequences=True for all layers except the last
- **Bidirectional RNNs**: Apply bidirectional wrappers when both past and future context is useful, such as in text processing
- **Dropout Regularisation**: Add dropout (20–50%) between layers to reduce overfitting. For LSTMs/GRUs, use recurrent_dropout to regularise recurrent connections

4. Choosing Between LSTM and GRU

- **LSTM:** Better for modelling long-term dependencies, thanks to separate forget, input, and output gates
- **GRU:** Faster to train and works well with smaller datasets or shorter sequences. With fewer gates, it's simpler and less prone to overfitting

Best Practices for RNNs

Tips For Effective Training

Training Tips

- **Learning Rate Scheduling:** Begin with a moderate rate (e.g., 0.001) and use a scheduler like ReduceLROnPlateau to lower it when validation performance stalls.
- **Gradient Clipping**: Apply clipnorm or clipvalue to control exploding gradients, a common issue in RNN training
- **Early Stopping**: Track validation loss and stop training if it fails to improve after a set number of epochs (patience), preventing overfitting and wasted computation

Stock Price Prediction

Stock Price Forecasting

Predicting Future Prices with LSTM and GRU

- Predicting stock prices is vital for investors and traders
- Uses historical stock data to forecast future closing prices
- Employs LSTM and GRU deep learning models
- Compares model performance and accuracy on unseen data

Stock Price Prediction

```
# Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
```

Stock Price Prediction

```
# Suppress warnings for clean output
import warnings
warnings.filterwarnings('ignore')

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Problem Statement
print("Problem Statement: Predict future stock prices using time series data with LSTM and GRU models.")

# Data Description
print("\nData Description:")
print("We are using a stock prices dataset containing 'Date' and 'Close' columns.")
print("Objective: Predict the closing price for the next day based on historical data.")

# Step 1: Load the Data
# Load the dataset and parse the 'Date' column as a datetime object
df = pd.read_csv('stock_prices.csv', parse_dates=['Date'], index_col='Date')
df = df[['Close']]  # Using only the 'Close' column for forecasting
print("\nFirst 5 rows of the dataset:")
```

Stock Price Prediction

```
# Step 2: Data Preprocessing
# Checking for missing values
print("\nChecking for missing values...")
print(df.isnull().sum())
```

Stock Price Prediction

```
# Scaling the data between 0 and 1 for better model performance
scaler = MinMaxScaler(feature_range=(0, 1))
df['Close'] = scaler.fit_transform(df[['Close']])
```

Stock Price Prediction

```
# Visualizing the scaled data
plt.figure(figsize=(10, 5))
plt.plot(df['Close'])
plt.title('Scaled Stock Prices')
plt.xlabel('Date')
plt.ylabel('Normalized Close Price')
plt.show()
```

Stock Price Prediction

```
# Step 3: Prepare the Data for Time Series Forecasting
# Function to create sequences of data for model training
def create_dataset(data, time_step=60):
    X, y = [ ], [ ]
    for i in range(len(data) - time_step - 1):
        X.append(data[i:(i + time_step), 0])
        y.append(data[i + time_step, 0])
    return np.array(X), np.array(y)
# Convert the dataframe to a numpy array
data = df.values
```

Stock Price Prediction

```
# Split data into training (80%) and testing sets (20%)
train_size = int(len(data) * 0.8)
test_size = len(data) - train_size
train_data, test_data = data[:train_size], data[train_size:]
```

Stock Price Prediction

```
# Create the training and testing datasets using a time step of 60
time_step = 60
X_train, y_train = create_dataset(train_data, time_step)
X_test, y_test = create_dataset(test_data, time_step)

# Reshape data to fit the input format expected by LSTM/GRU models
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
```

Stock Price Prediction

```
# Step 4: LSTM Model Implementation
print("\nBuilding and Training the LSTM Model...")

# Define the LSTM model
lstm_model = Sequential()
lstm_model.add(LSTM(64, return_sequences=True, input_shape=(time_step, 1)))
lstm_model.add(Dropout(0.2))  # Dropout to prevent overfitting
lstm_model.add(LSTM(32))
lstm_model.add(Dense(1))  # Output layer with one neuron for prediction
lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# Train the LSTM model
history_lstm = lstm_model.fit(X_train, y_train, epochs=20, batch_size=32,
validation_data=(X_test, y_test), verbose=1)
```

Stock Price Prediction

```
# Define the GRU model
gru_model = Sequential()
gru_model.add(GRU(64, return_sequences=True, input_shape=(time_step, 1)))
gru_model.add(Dropout(0.2))  # Dropout to prevent overfitting
gru_model.add(GRU(32))
gru_model.add(Dense(1))  # Output layer with one neuron for prediction
gru_model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# Train the GRU model
history_gru = gru_model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test,
y_test), verbose=1)
```

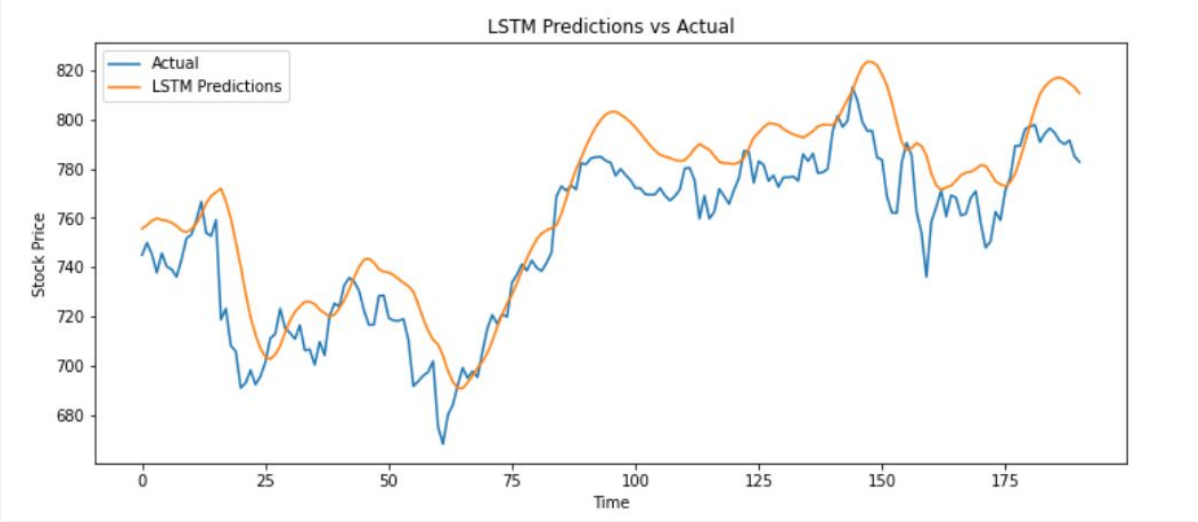
Stock Price Prediction

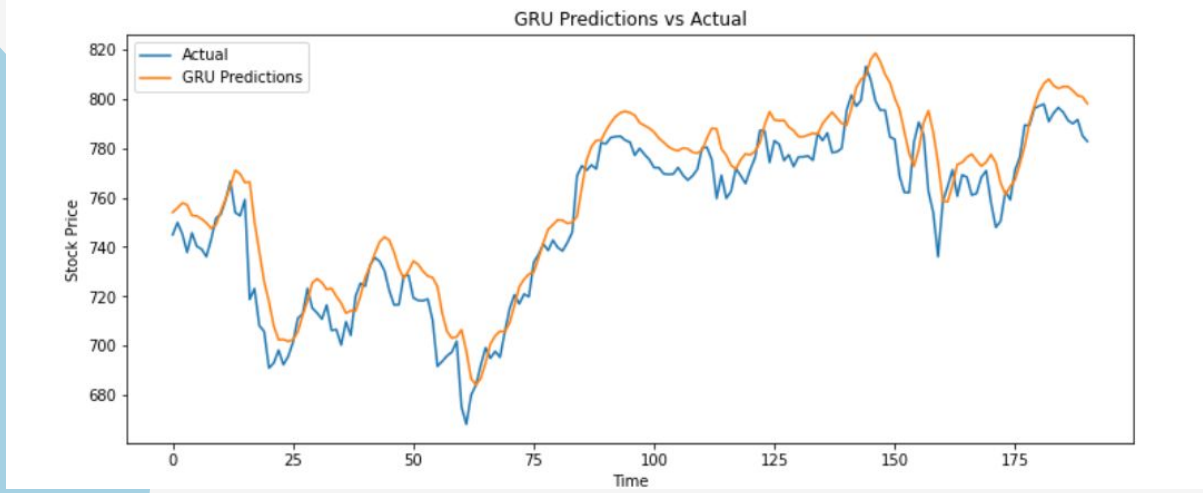
```
# Step 6: Model Evaluation

# Function to evaluate and plot the model's performance
def evaluate_model(model, X_train, y_train, X_test, y_test, model_name):
    # Make predictions on training and testing data
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    # Inverse transform predictions to original scale
    train_pred = scaler.inverse_transform(train_pred)
    test_pred = scaler.inverse_transform(test_pred)
    y_train = scaler.inverse_transform(y_train.reshape(-1, 1))
    y_test = scaler.inverse_transform(y_test.reshape(-1, 1))

    # Calculate evaluation metrics: RMSE and MAE
    train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
    test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))
    train_mae = mean_absolute_error(y_train, train_pred)
    test_mae = mean_absolute_error(y_test, test_pred)
```

Stock Price Prediction

```
print(f"\n{model_name} - Train RMSE: {train_rmse:.4f}, Test RMSE: {test_rmse:.4f}")
    print(f"{model_name} - Train MAE: {train_mae:.4f}, Test MAE: {test_mae:.4f}")

    # Plotting actual vs predicted values
    plt.figure(figsize=(12, 5))
    plt.plot(y_test, label='Actual')
    plt.plot(test_pred, label=f'{model_name} Predictions')
    plt.title(f'{model_name} Predictions vs Actual')
    plt.xlabel('Time')
    plt.ylabel('Stock Price')
    plt.legend()
    plt.show()
```

Stock Price Prediction





Stock Price Prediction

```
# Evaluate the LSTM Model
print("\nEvaluating LSTM Model...")
evaluate_model(lstm_model, X_train, y_train, X_test, y_test, "LSTM")

# Evaluate the GRU Model
print("\nEvaluating GRU Model...")
evaluate_model(gru_model, X_train, y_train, X_test, y_test, "GRU")
```

Q&A Session

- Open floor for questions and clarifications.